

04

깃허브 시작하기

04-1 원격 저장소와 깃허브

04-2 깃허브 가입하기

04-3 지역 저장소를 원격 저장소에 연결하기

04-4 지역 저장소와 원격 저장소 동기화하기

04-5 깃허브에 SSH 원격 접속하기



원격 저장소란

- 깃에서는 지역 저장소와 원격 저장소(remote repository)를 연결해서 버전 관리하는 파일을 쉽게 백업할 수 있음
원격 저장소는 지역 저장소가 아닌 컴퓨터나 서버에 만든 저장소를 말함
- 원격 저장소는 지역 저장소와 연결되어 있으면서 '백업'과 '협업'이라는 중요한 역할을 함
특히 규모가 큰 프로젝트를 진행한다면 다른 사람과 협업할 때가 많은데, 이때 원격 저장소의 역할은 더욱 중요해짐
- 원격 저장소를 직접 구축할 수도 있지만 만들고 유지하는 것이 쉽지 않으므로 원격 저장소를 제공하는 서비스를 주로 사용
그중에서 깃과 관련해 가장 많이 사용하는 서비스가 바로 깃허브!

깃허브로 할 수 있는 일들 - (1)

원격 저장소에서 깃을 사용할 수 있습니다

- 깃허브는 깃 사용을 위한 원격 저장소를 제공하는 서비스이므로 따로 깃을 설치하지 않고도 온라인에서 깃의 버전 관리 기능을 사용할 수 있음
- 지역 저장소를 만들지 않아도 깃허브에 원격 저장소를 만들어 사용할 수도 있고, 지역 저장소가 있다면 원격 저장소와 연결해서 사용할 수도 있음

지역 저장소를 백업할 수 있습니다

- 깃허브에 원격 저장소를 만들고 사용자 컴퓨터의 지역 저장소를 연결한 후 동기화하면 지역 저장소를 인터넷에서 백업할 수 있음
- 깃허브가 아닌 구글 드라이브 같은 클라우드 디스크에 백업할 수도 있지만 깃허브에 백업하면 원격 저장소에 손쉽게 커밋할 수 있음

깃허브로 할 수 있는 일들 - (2)

온라인 개발 툴을 사용할 수 있습니다

- 깃허브에 코드스페이스(Codespaces)라는 새로운 기능이 추가되어 클라우드에서 소스를 작성하고 편집할 수 있음
- 컴퓨터가 바뀌거나 개발 환경이 달라질 때마다 VS Code를 설치하고 필요한 확장 기능을 추가하는 과정을 반복해야 하지만, 코드스페이스를 사용하면 깃허브에 나만의 개발 환경을 만들어 놓고
- 언제든지 온라인에서 VS Code 편집기를 열어 수정하고 커밋할 수 있음
- 지역 저장소를 만들고 깃허브로 올리는(push) 과정도 필요 없음

협업 프로젝트에 사용할 수 있습니다

- 팀 프로젝트를 진행할 때도 이전 깃허브가 기본 저장소가 되고 있음
- 원격 저장소이므로 인터넷만 가능하면 누구나 접근할 수 있고, 깃과 깃허브에서 여러 가지 협업 도구를 제공하므로 깃허브를 사용하면 팀원 여러 명이 하나의 프로젝트를 진행하기도 쉬움

깃허브로 할 수 있는 일들 - (3)

자신의 개발 이력을 남길 수 있습니다

- 깃허브에서 소스를 수정하고 오픈 소스에 참여해서 하는 일은 사용자 초기 화면에 날짜별로 모두 기록으로 남음
- 뽀뽀하게 기록된 것을 보면 스스로 성실하게 개발했다는 뿌듯함을 느끼기도 함
- 최근에는 개발자를 뽑을 때 깃허브 계정을 요구하는 곳이 있음
지원자가 어떤 주제에 관심이 많은지, 어떤 것을 개발했는지, 그리고 무엇을 개발하는지 한눈에 확인할 수 있기 때문
- 깃허브는 개발자가 자신의 개발 이력을 관리하기 좋은 플랫폼

다른 사람의 소스를 살펴볼 수 있고, 오픈 소스에 참여할 수도 있습니다

- 개발자로서 실력을 높이는 방법은 다른 사람의 소스를 읽어 보고 분석하면서
자기 나름대로 소스를 수정하고 작성해 보는 것
- 깃허브에는 전세계 개발자들이 공개해 놓은 소스들이 많아 이 소스를 얼마든지 내 저장소로 가져와서 분석해 볼 수 있음
- 깃허브에는 깃을 비롯해 웹 개발이나 인공지능, 데이터 과학 등 전 개발 분야에 걸쳐 다양한 오픈 소스가 등록되어 있음
이러한 오픈 소스를 살펴보고 참여할 수 있는 것도 깃허브의 커다란 매력

깃허브에 가입하기

1. www.github.com에 접속한 후 [Sign up]을 클릭
2. 깃허브에서 사용할 이메일 계정을 입력한 후 [Continue]를 클릭
3. 중복되는 메일 계정이 없으면 단계별로 비밀번호(password), 사용자 이름(username), 업데이트 소식을 받을지 여부를 묻는데 단계별로 필요한 정보를 입력하고 [Continue]를 클릭
사용자 이름은 영문자와 숫자만 사용할 수 있고, 단어가 2개이면 붙임표(-)로 연결할 수도 있음
4. 사용자 정보를 모두 입력하면, 'Verify your account'에 있는 문제를 풀고 [Create account]를 클릭
5. 깃허브 계정을 만들 때 사용한 이메일 주소로 론치 코드(launch code)가 도착하는데,
이 코드를 확인해서 깃허브 화면에 입력해야 계정 만들기가 끝남
6. 혼자 사용하는지, 여러 명이 함께 사용하는지 선택한 후 [Continue]를 클릭
이후에 몇 가지 선택 사항이 나타나는데 그냥 [Continue]를 클릭해도 됨
7. 깃허브에는 무료인 개인 계정과 유료인 팀 계정이 있는데, 깃허브를 공부하거나 개인 프로젝트라면 개인 계정을 사용하면 됨
[Continue for free]를 클릭하는 것으로 계정 등록이 끝남
8. 깃허브 화면을 처음 경험한다면 꽤 복잡해 보이지만, 아직 아무 저장소도 만들지 않았으므로 처음에는 깃허브 사용법과 깃허브의 최근 변경 사항 내용이 보임

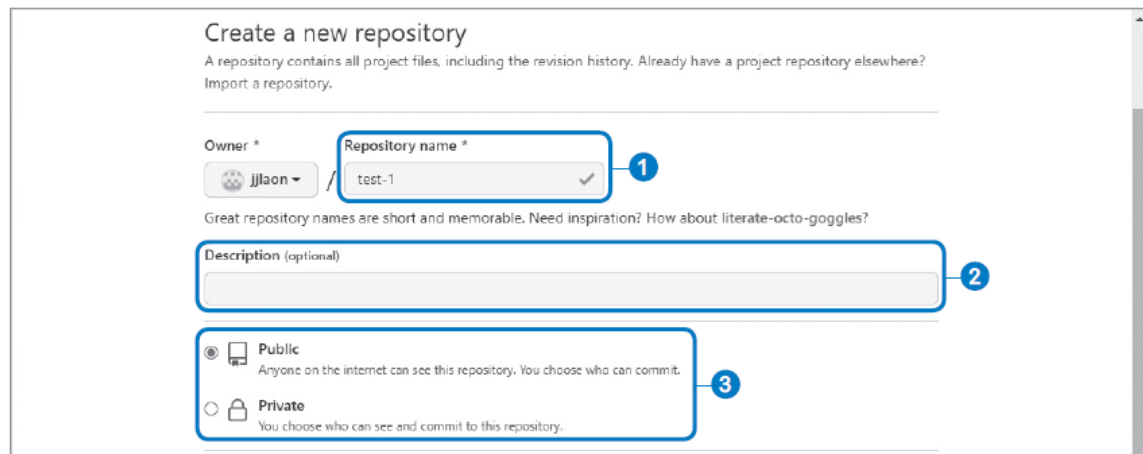
지역 저장소와 원격 저장소

- 깃허브에서 버전을 관리할 때도 저장소를 만들어 사용해야 함
- 사용자 컴퓨터에 있는 저장소는 '지역 저장소(local repository)'라 하고, 깃허브에 있는 저장소는 '원격 저장소(remote repository)'라고 함
- 우선 지역 저장소를 만들어 작업한 후 그 내용을 원격 저장소로 올리고 변경 사항이 생길 때마다 원격 저장소에도 반영할 것
- 지역 저장소에서 원격 저장소로 커밋을 등록하는 것을 '푸시(push)'라고 함
지역 저장소를 거치지 않고 원격 저장소에서 커밋을 만들 수도 있는데, 원격 저장소의 변경 사항을 지역 저장소로 내려받는 것을 '풀(pull)'이라고 함
- 지역 저장소와 원격 저장소를 연결해 놓았기 때문에 지역 저장소의 변경 사항은 항상 원격 저장소로 올려 두어야 하고, 원격 저장소에서 무언가 변경 사항이 있다면 지역 저장소에 내려받아 두어야 함
- 이렇게 지역 저장소와 원격 저장소를 항상 같게 유지하는 것을 '동기화(synchronize)'라고 함

원격 저장소 만들기 - (1)

1. 깃허브에 로그인한 후 화면 오른쪽 위에 있는 [+]를 클릭하고 [New repository]를 선택
2. 저장소 이름을 비롯해서 필요한 항목을 기입하고 [Create repository]를 클릭

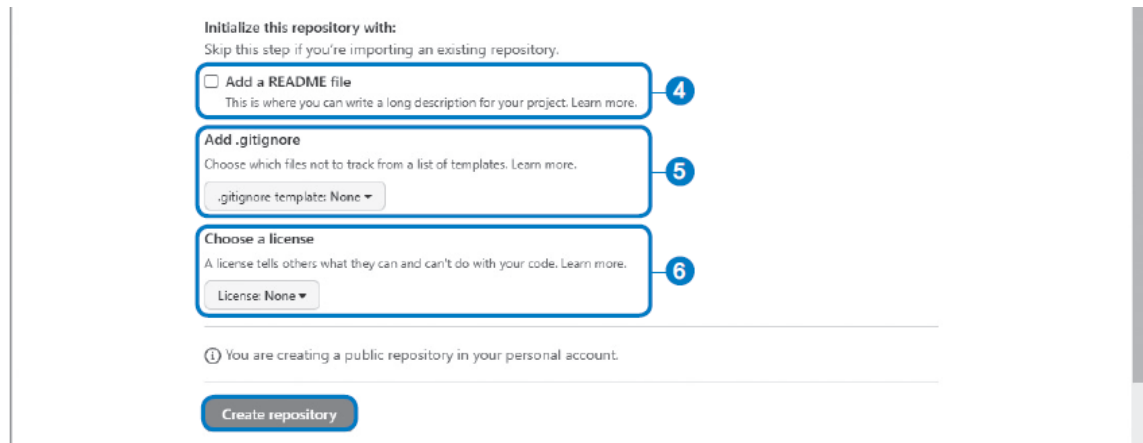
- ① **Repository name:** 저장소 이름을 입력
영문과 숫자, 언더바(_), 붙임표(-) 등을 사용할 수 있음
공백이 포함되어 있으면 자동으로 붙임표(-)로 바꿈
- ② **Description:** 저장소를 소개하는 간단한 설명을 입력
이 부분은 옵션이므로 반드시 입력하지 않아도 됨
- ③ **Public / Private:** 저장소를 공개로 할지 비공개로 할지 선택
공개 저장소는 주소만 알면 누구나 볼 수 있으나 만일 다른 사람에게 보여 주고 싶지 않은 프로젝트를 관리한다면 저장소를 만들 때 비공개(private)로 하면 됨



The screenshot shows the 'Create a new repository' page on GitHub. It includes a title, a subtitle, and a link to 'Import a repository'. Below this, there are three main sections: 'Owner', 'Repository name', and 'Description'. The 'Owner' section shows a dropdown menu with 'jjlaon' selected. The 'Repository name' section has a text input field with 'test-1' and a checkmark icon. The 'Description' section has a text input field. Below the description, there are two radio button options: 'Public' (selected) and 'Private'. The 'Public' option has a description: 'Anyone on the internet can see this repository. You choose who can commit.' The 'Private' option has a description: 'You choose who can see and commit to this repository.' Numbered blue circles (1, 2, 3) are placed next to the 'Repository name' field, the 'Description' field, and the 'Public/Private' options respectively.

원격 저장소 만들기 - (2)

- ④ **Add a README:** 저장소를 소개하고 설명하는 내용을 작성하는 README 파일을 자동으로 만들려면 체크
여기에서는 체크하지 않도록 함
- ⑤ **Add .gitignore:** .gitignore 파일은 지역 저장소의 파일을 깃허브의 저장소로 푸시할 때 제외할 파일을 지정해 놓은 것 [▼]를 클릭한 후 어떤 언어와 관련된 것을 .gitignore 파일에 지정할지 선택
예를 들어 C++을 선택한다면 C++에서 사용하는 컴파일된 라이브러리나 실행 파일을 깃에서 무시하도록 .gitignore 파일을 자동으로 만들어 줌
- ⑥ **Choose a license:** 오픈 소스 프로젝트를 위한 저장소를 만들 때 해당 오픈 소스의 라이선스를 선택



The screenshot shows the GitHub 'Initialize this repository' screen. It has three main sections, each with a blue box and a circled number on the right:

- 4 Add a README file:** Includes a checkbox and a text area for a project description. A link 'Learn more.' is present.
- 5 Add .gitignore:** Includes a dropdown menu for '.gitignore template: None'. A link 'Learn more.' is present.
- 6 Choose a license:** Includes a dropdown menu for 'License: None'. A link 'Learn more.' is present.

Below these sections, there is a note: 'You are creating a public repository in your personal account.' and a 'Create repository' button.

원격 저장소 만들기 - (3)

3. 원격 저장소가 만들어지면서 즉시 저장소 페이지로 이동
아직 아무 파일도 들어 있지 않고, 맨 위에는 저장소에 접속하는 방법이 나타남
저장소에 접속할 때는 HTTPS 방식이나 SSH 방식 중에서 선택할 수 있는데 우선 우리는 HTTPS 방식을 사용해 접속할 것
4. 화면에 나타난 HTTPS 주소를 사용해 언제든지 깃허브 저장소에 접속할 수도 있고 파일을 올릴 수 있음
깃허브 원격 저장소 주소만 알고 있다면 어디에서든 지역 저장소를 백업하거나 다른 사람과 협업할 수 있음

원격 저장소의 HTTPS 주소는 다음과 같은 형태

```
https://github.com/아이디/저장소명
```

예를 들어 깃허브 계정이 jump2dev이고, 저장소 이름이 test-1이라면 주소는 다음과 같음

```
https://github.com/jump2dev/test-1
```

지역 저장소 만들기

1. 홈 디렉터리에 loc-git이라는 새 디렉터를 만들면서도 동시에 지역 저장소로 지정
그리고 loc-git 디렉터리 안에 f1.txt 문서를 만듦

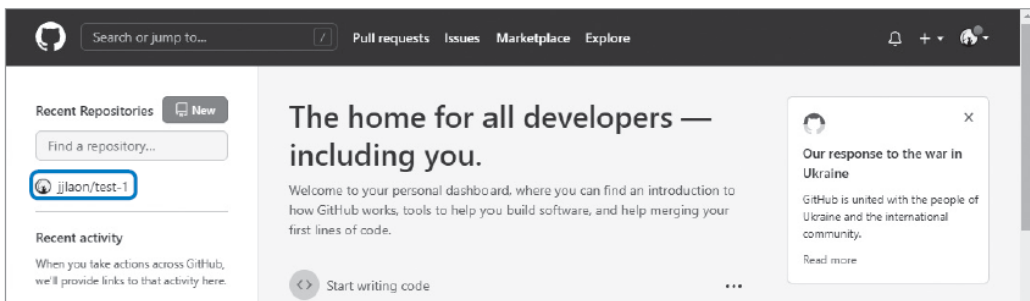
```
$ cd ~  
$ git init loc-git  
$ cd loc-git  
$ vim f1.txt
```

2. f1.txt에는 간단하게 영문자 'a'만 입력하고 파일을 저장한 후 편집기를 종료
3. f1.txt를 스테이지에 올린 후 커밋하고 커밋 메시지는 'add a'라고 함
git log 명령으로 커밋이 잘 되었는지 확인하고 아직 터미널 창은 닫지 않기

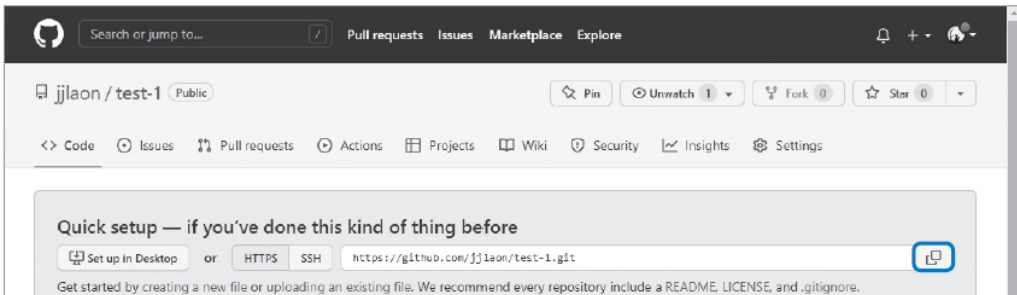
```
$ git add f1.txt  
$ git commit -m "add a"  
$ git log
```

원격 저장소에 연결하기 - (1)

1. 지역 저장소와 원격 저장소를 연결하려면 우선 원격 저장소의 HTTPS 주소를 알아야 함
웹 브라우저에서 깃허브에 로그인하면 화면 왼쪽에 저장소 목록이 나타남
지역 저장소와 연결할 저장소를 클릭



2. 깃허브 주소 오른쪽에 있는  을 클릭하면 주소가 복사



원격 저장소에 연결하기 - (2)

3. 저장소 주소를 복사했다면 터미널 창에 다음과 같이 입력
이 명령은 원격 저장소(remote)에 origin을 추가(add)하겠다고 깃에게 알려 주는 것
여기에서 origin은 깃허브 저장소 주소(https://github.com/...)를 가리킴
깃허브 저장소 주소를 그대로 쓰면 너무 길기 때문에 origin이라는 단어로 줄여서 remote에 추가하는 것
이렇게 지역 저장소를 원격 저장소에 연결하는 것은 한 번만 하면 됨

```
$ git remote add origin 복사한 주소 붙여넣기
```

4. 오류 메시지 없이 프롬프트(\$)가 나타나면 제대로 연결된 것
5. 원격 저장소(remote)에 제대로 연결됐는지 확인하려면 다음처럼 git remote 명령에 -v 옵션을 붙여서 입력

```
$ git remote -v
```

6. remote에 origin이 연결되어 있고, origin이 가리키는 주소가 바로 옆에 표시됨
주소 끝에 있는 (fetch)와 (push)는 앞으로 배울 것이므로 여기에서는 지역 저장소가 원격 저장소에 잘 연결되었는지만 확인

처음으로 원격 저장소에 커밋 올리기 - (1)

1. 지역 저장소 디렉터리에서 터미널 창에 다음과 같이 입력

지역 저장소의 브랜치를 origin(원격 저장소)의 main 브랜치로 푸시하라는 명령

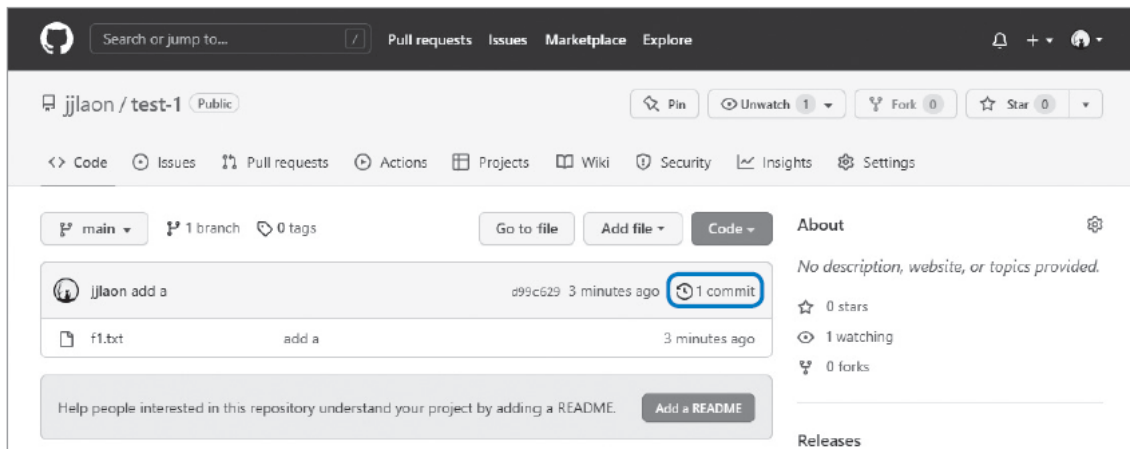
여기에서 -u 옵션은 지역 저장소의 브랜치를 원격 저장소의 브랜치에 연결하기 위한 것으로 처음에 한 번만 사용하면 됨
이후에는 -u 옵션이나 main 브랜치 이름 없이 간단히 푸시할 수 있음

```
$ git push -u origin main
```

2. 처음으로 원격 저장소에 푸시할 때는 깃허브 로그인 창이 나타남
[Sign in with your browser]를 클릭
3. 깃을 사용한 지역 저장소와 깃허브 저장소를 연결하기 위해 [Authorize GitCredential Manager]를 클릭
4. 깃허브 계정의 비밀번호를 입력하고 [Confirm password]를 클릭하면 사용자 인증이 끝남
이제부터는 사용자 인증 없이 깃허브 저장소에 푸시할 수 있음

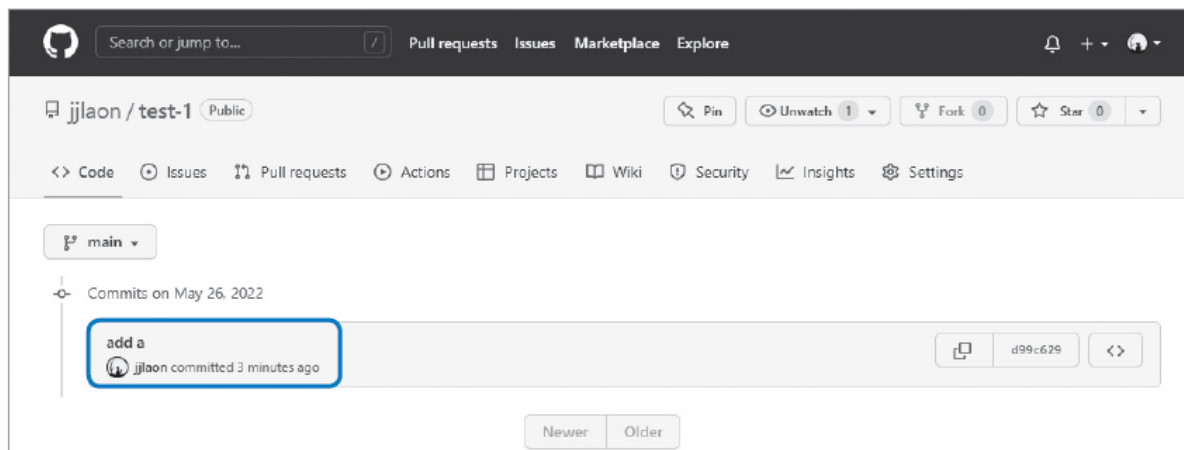
처음으로 원격 저장소에 커밋 올리기 - (2)

5. 사용자 인증이 끝나는 것과 동시에 터미널 창에서는 푸시가 진행됨
푸시가 끝나면 프롬프트(\$)가 나타남
6. 푸시가 끝났다는 것은 지역 저장소의 커밋이 원격 저장소로 올라갔다는 뜻
푸시가 끝났으면 깃허브 저장소가 열려 있는 웹 브라우저 창으로 돌아와 {{ F5 }}를 눌러 새로 고침
지역 저장소에 있던 f1.txt 파일이 원격 저장소에 올라와 있음
저장소의 파일 목록에는 파일 이름과 함께 커밋 메시지도 나타남
파일 목록 오른쪽 위에 있는 [1 commit]을 클릭



처음으로 원격 저장소에 커밋 올리기 - (3)

7. 커밋한 날짜와 사람, 메시지 등을 볼 수 있음



원격 저장소에 파일 올리기 — git push - (1)

1. 지역 저장소에서 또 다른 커밋을 만들고 다시 푸시
빔으로 f1.txt을 다시 열고, 원래 내용 다음 줄에 'b'를 추가하고 저장

```
$ vim f1.txt
```

2. 다음 명령을 사용해 스테이징과 커밋을 한꺼번에 실행
git commit 명령에서 -am은 스테이징 옵션(-a)과 메시지 옵션(-m)을 함께 쓴 것으로,
최소한 한 번이라도 커밋한 파일(tracked 파일)이어야 사용할 수 있음
커밋 메시지는 'add b'라고 함

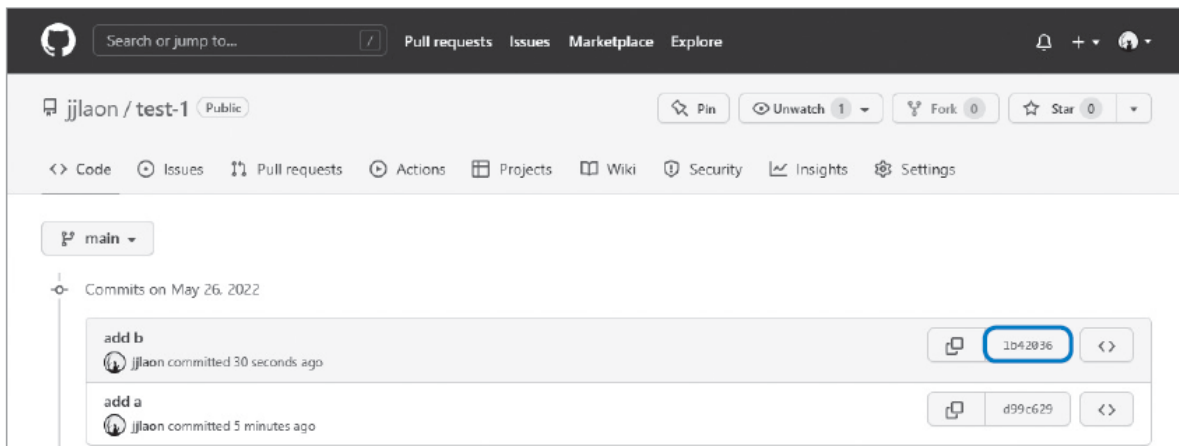
```
$ git commit -am "add b"
```

3. 이미 앞에서 원격 저장소에 푸시하면서 사용자 인증을 했으므로 이제부터 파일을 푸시할 때는 git push라고만 입력하면 됨

```
$ git push
```

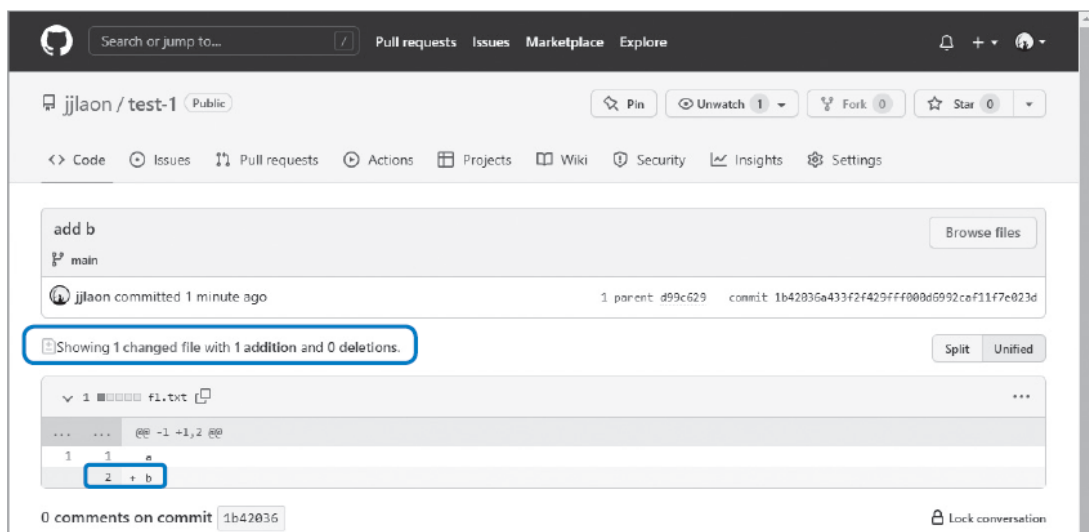
원격 저장소에 파일 올리기 — git push - (2)

4. 방금 만든 커밋이 깃허브의 원격 저장소로 푸시
5. 웹 브라우저에서 깃허브 저장소 화면을 새로 고침
f1.txt 파일을 수정한 것이기 때문에 파일 목록에 계속 f1.txt 파일만 보임
그런데 파일 목록 위를 보면 1 commit이었던 것이 2 commits으로 바뀌었음
[2 commits]을 클릭
6. 조금 전에 푸시한 add b 커밋이 올라온 것을 볼 수 있음
add b 커밋이 어떤 걸 변경한 것인지 궁금하다면 커밋 이름 오른쪽에 있는 커밋 해시를 클릭



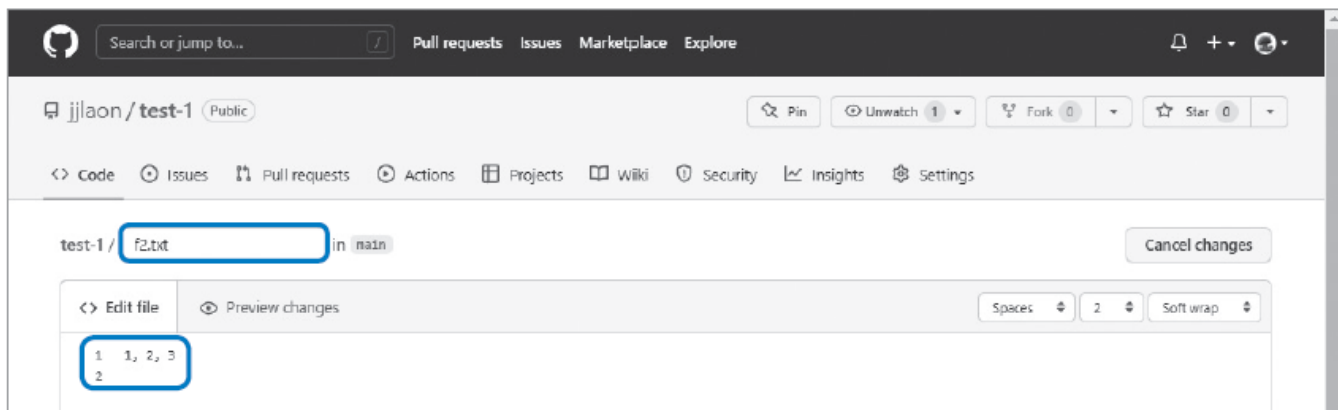
원격 저장소에 파일 올리기 — git push – (3)

7. 하나의 파일이 변경됐고, 추가된 것은 하나, 삭제된 것은 없다고 표시됨
그리고 그 아래를 보면 초록색이 추가된 부분이 있는데 '+b'라고 되어 있음
파일에 b가 추가되었다는 뜻



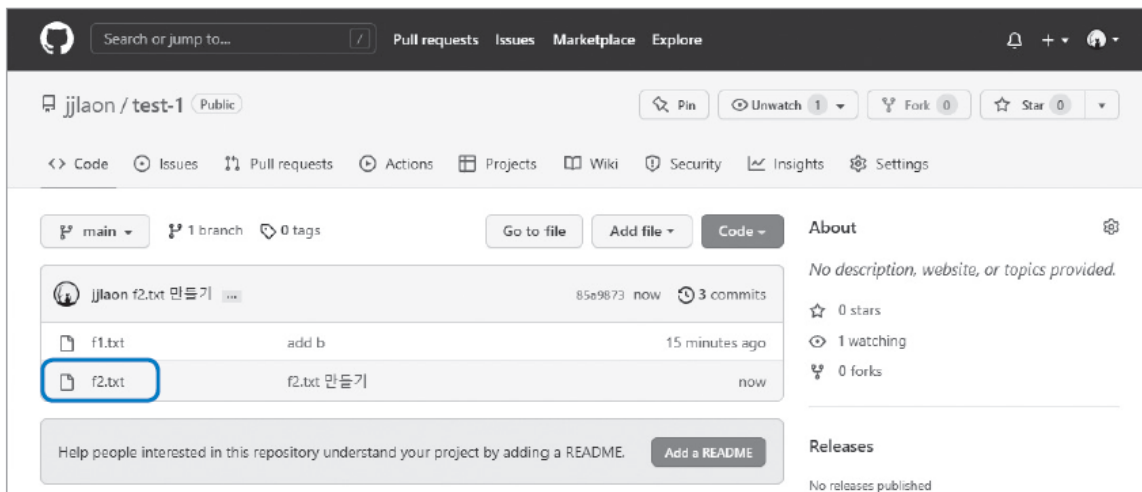
원격 저장소에서 직접 커밋하기 - (1)

1. 앞에서 커밋을 푸시한 원격 저장소로 접속
앞에서 푸시한 f1.txt 파일이 있음
여기에 새로운 파일을 추가
파일 목록 위에 있는 [Add file]을 클릭한 후 [Create new file]을 선택
2. 맨 위에 파일 이름을 입력한 후 내용을 작성
여기에서는 파일 이름을 f2.txt로 하고, 내용에는 숫자 1, 2, 3을 입력



원격 저장소에서 직접 커밋하기 - (2)

3. 파일 내용을 입력했다면 화면 아래로 이동
기본적인 커밋 메시지(f2.txt 만들기)가 입력되어 있음
이 메시지를 그냥 사용하거나 원하는 내용으로 수정한 후 [Commit new file]을 클릭
4. 원격 저장소에 새로운 커밋이 추가되었음



원격 저장소에서 커밋 내려받기 — `git pull` - (1)

1. 앞에서 loc-git 지역 저장소를 원격 저장소에 연결한 후 푸시했고 깃허브 사이트에서 바로 f2.txt라는 파일을 새로 만들었음
그러므로 loc-git 지역 저장소에는 아직 f2.txt 파일이 없음
터미널 창에서 loc-git 디렉터리로 이동한 후 `ls` 명령을 사용해 디렉터리 안의 내용을 확인해 보면 지역 저장소에서 만든 f1.txt 파일만 있음

```
$ ls
```

2. 원격 저장소에서 커밋을 풀할 때는 `git pull` 명령을 사용함
그리고 그 뒤에 원격 저장소 이름과 지역 저장소의 브랜치 이름을 넣어 줌
여기에서는 origin(원격 저장소)을 지역 저장소의 main 브랜치로 가져오라고 할 것

```
$ git pull origin main
```

3. 원격 저장소에서 소스 파일을 가져오는 과정이 화면에 나타남
\$가 화면에 표시되면 가져오기가 끝난 것

원격 저장소에서 커밋 내려받기 — `git pull` - (2)

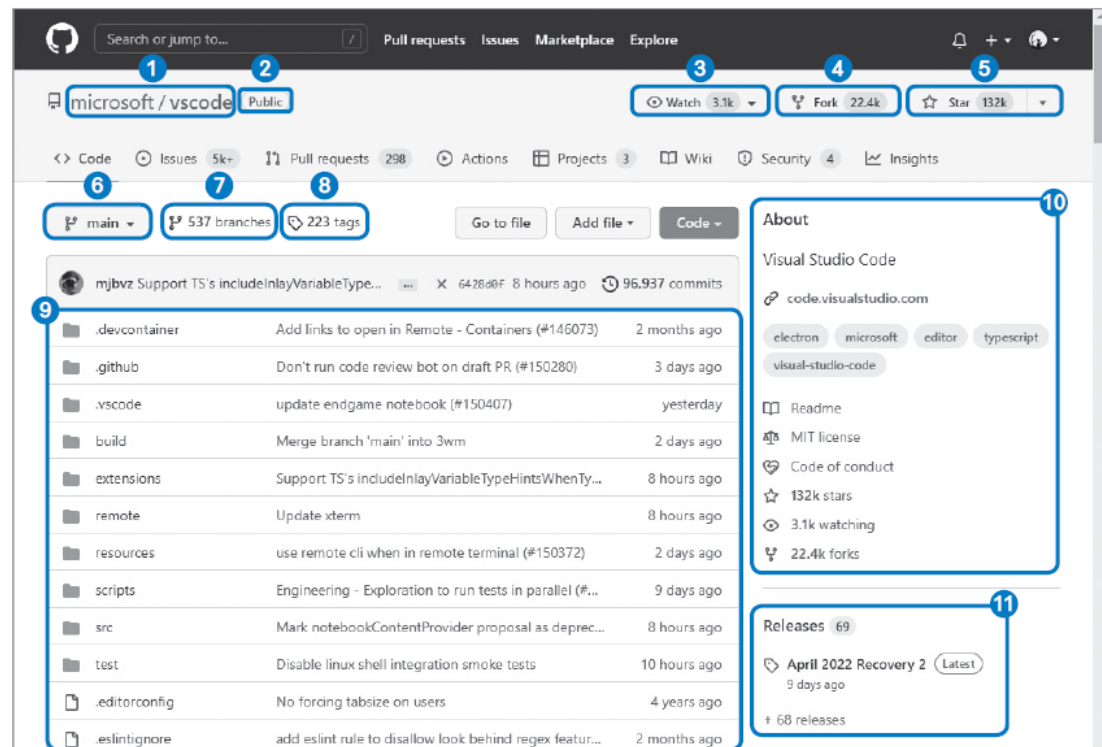
4. `git log` 명령으로 커밋 로그를 보면 깃허브 사이트에서 만들었던 커밋이
지역 저장소 커밋 로그에도 나타나는 것을 확인할 수 있음
그리고 커밋 해시 오른쪽을 보면 (HEAD -> main) 외에 origin/main도 있음
원격 저장소의 최신 커밋이라는 뜻

깃허브 원격 저장소 화면 살펴보기 - (1)

원격 저장소 화면에는 여러 가지 정보가 담겨 있는데 여기에 있는 정보와 기능을 모두 사용하지는 않겠지만, 최소한 어떤 정보와 기능이 있는지 오픈 소스인 VS Code 저장소 화면을 예로 들어 설명

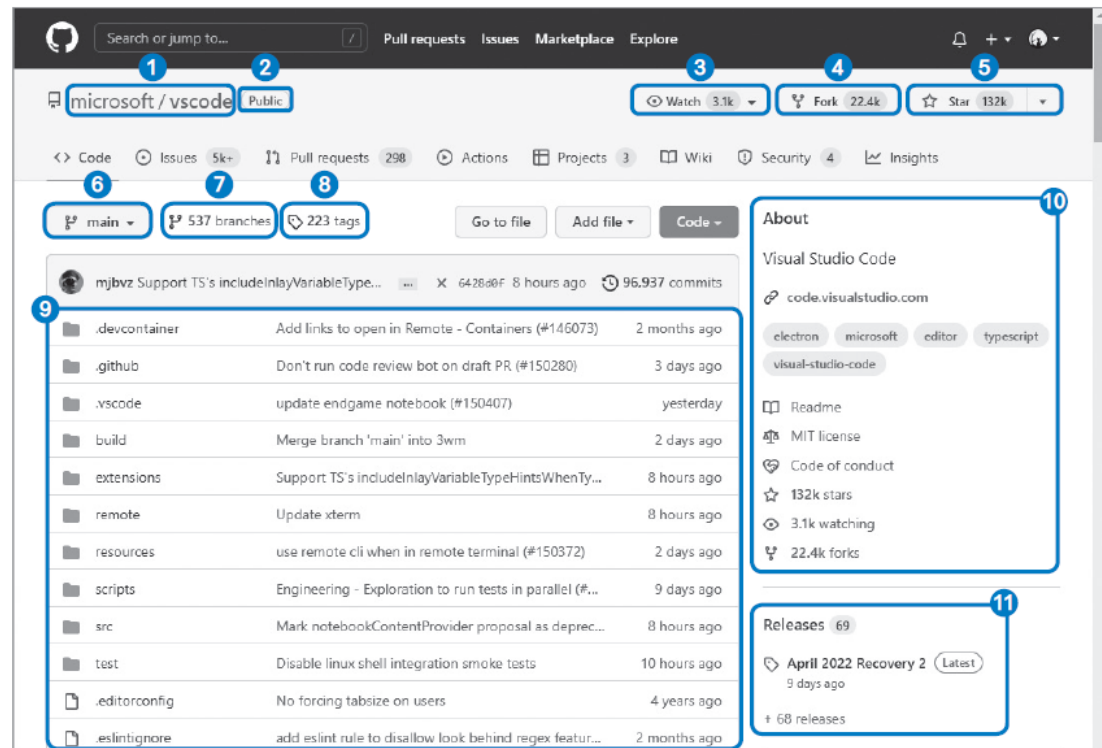
- ① 깃허브 계정/저장소 이름
계정을 클릭하면 이 계정의 요약 정보 화면으로 이동
저장소의 다른 화면에 있더라도 저장소 이름을 클릭하면 저장소 파일 목록 화면으로 이동
- ③ Watch: 저장소에 대한 알림을 받는 사용자의 수를 표시
- ④ Fork: 현재 저장소를 자신의 계정으로 복제한 사용자의 수를 표시
- ⑤ Star: 현재 저장소를 즐겨찾기에 추가한 사용자의 수를 표시

다른 저장소의 소스를 살펴볼 때 ③, ④, ⑤ 등의 개수를 눈여겨 보면 해당 저장소의 소스를 참고할지 결정하는 데 도움이 됨



깃허브 원격 저장소 화면 살펴보기 - (2)

- ⑥ 현재 브랜치를 나타냄
브랜치가 여럿이라면 이 목록을 클릭한 후 브랜치를 전환할 수 있음
- ⑦ 현재 저장소의 브랜치 개수를 표시
- ⑧ 태그(커밋을 이해하기 쉽게 설명을 붙인 것)를 사용한 커밋의 수를 표시
- ⑨ 저장소의 파일 목록이 나타남
- ⑩ 저장소를 소개하는 요약 정보가 표시
- ⑪ 깃허브를 통해 소프트웨어를 제공한다면 릴리즈 버전과 패키지를 만들어 그 내용을 나열



저장소 화면 오른쪽에 나타나는 저장소 관련 정보를 입력하려면 About 오른쪽의 ⚙️ 를 클릭한 후 필요한 내용을 입력
릴리즈나 패키지 항목이 필요하지 않으면 체크 표시를 지워서 화면에서 감출 수 있고 [Save changes]를 클릭하면 내용이 저장됨

SSH 원격 접속이란

- SSH는 secure shell의 줄임말로 보안이 강화된 안전한 방법으로 정보를 교환하는 방식
- SSH에서는 기본적으로 프라이빗 키(private key)와 퍼블릭 키(public key)를 한 쌍으로 묶어서 컴퓨터를 인증
- 퍼블릭 키는 말 그대로 외부로 공개되고, 프라이빗 키는 아무도 알 수 없게 사용자 컴퓨터에만 저장됨
사용자 컴퓨터에서 SSH 키 생성기를 실행하면 프라이빗 키와 퍼블릭 키가 만들어짐
- 일반적으로 깃허브의 원격 저장소에 파일을 올리는 등의 작업을 하려면 아이디와 비밀번호를 입력해서 깃허브에게 자신이 해당 저장소를 만든 계정의 주인임을 인증해야 함
- 이에 비해 SSH 원격 접속은 프라이빗 키와 퍼블릭 키를 사용해 현재 사용하는 기기를 깃허브에 인증하는 방식
- 예를 들어 서버 환경에서 깃허브 저장소에 접속해야 한다면 서버 자체를 깃허브에 등록하고,
개인 노트북으로 접속한다면 노트북을 깃허브에 등록해 둬
이렇게 하면 터미널 창에서 따로 인증하지 않아도 깃허브에 접속할 수 있음
- 터미널 창에서 깃허브를 사용하다 보면 아이디와 비밀번호를 요구하는 경우가 많은데,
SSH 접속 방법을 사용하면 자동 로그인 기능을 통해 이러한 번거로움을 줄일 수 있음

SSH 키 생성하기 – (1)

1. 터미널 창에서 홈 디렉터리로 이동하고 'ssh-keygen'이라고 입력
화면에 SSH 키가 저장되는 디렉터리 경로가 표시되면서 파일 이름을 입력하라고 함
SSH 키가 저장되는 디렉터리는 홈 디렉터리 안에 있는 .ssh 디렉터리임을 확인할 수 있음
파일 이름은 입력하지 말고 {{ Enter }}를 누름

```
$ cd ~  
$ ssh-keygen
```

2. {{ Enter }}를 두 번 더 누르면 화면에 SSH 접속을 위한 비밀번호가 만들어짐
실제로 내부를 들여다보면 비밀번호가 매우 복잡해서 외부에서 쉽게 공격할 수 없음
화면에는 몇 가지 파일 경로가 표시되는데 그중에 id_rsa 파일이 프라이빗 키이고, id_rsa.pub 파일이 퍼블릭 키

SSH 키 생성하기 – (2)

3. 방금 만든 키들이 정말 .ssh 디렉터리에 저장되었는지 확인

.ssh 디렉터리는 홈 디렉터리 하위에 만들어지므로 터미널 창에 다음과 같이 입력해서 .ssh 디렉터리로 이동한 후 그 안의 내용을 살펴봄

```
$ cd .ssh  
$ ls -la
```

4. .ssh 디렉터리 안에 프라이빗 키인 id_rsa 파일과 퍼블릭 키인 id_rsa.pub 파일이 만들어진 것을 확인할 수 있음

깃허브에 퍼블릭 키 전송하기 - (1)

1. SSH로 접속하려면 우선 만들어 놓은 퍼블릭 키를 깃허브에 올려야 함
다음처럼 clip 명령을 사용해서 퍼블릭 키가 담겨 있는 id_rsa.pub 파일의 내용을 복사

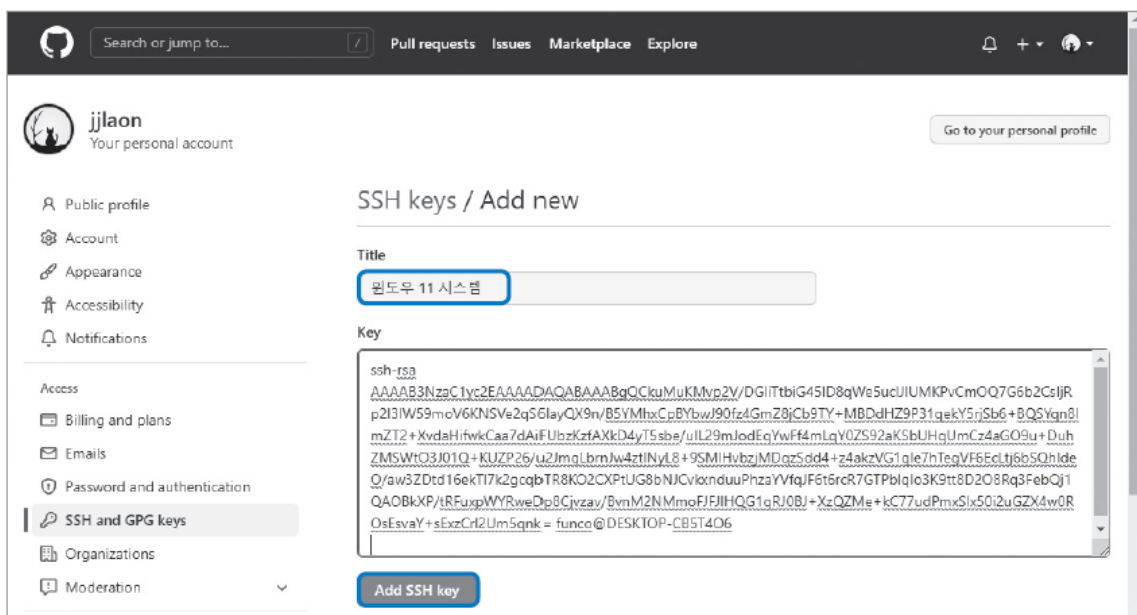
```
$ clip < ~/.ssh/id_rsa.pub
```
2. 브라우저에서 깃허브에 접속한 후 로그인
그리고 화면 오른쪽 위에 있는 사용자 아이콘을 클릭한 후 [Settings]를 선택
3. 여러 가지 설정 항목이 나타나는데 Access 카테고리에서 [SSH and GPG keys]를 선택
아직 아무 SSH 키도 등록되어 있지 않은 상태
새로운 퍼블릭 키를 추가하기 위해 화면 오른쪽에 나타난 [New SSH key]를 클릭

깃허브에 퍼블릭 키 전송하기 - (2)

4. SSH에는 퍼블릭 키를 여러 개 등록할 수 있음

현재 등록하는 SSH 퍼블릭 키를 쉽게 알아볼 수 있도록 Title 항목에 제목을 붙임

맥과 윈도우 시스템에서 사용할 SSH 키를 등록할 것이므로 여기에서는 Title에 윈도우 시스템 11을 사용할 것이라고 작성
그리고 Key 항목에 앞에서 복사한 퍼블릭 키값을 붙여 넣고 [Add SSH key]를 클릭해서 SSH 키를 추가



깃허브에 퍼블릭 키 전송하기 - (3)

5. 퍼블릭 키를 추가할 때 비밀번호를 확인함
깃허브 비밀번호를 입력한 후 [Confirm password]를 클릭
6. 만들었던 SSH 퍼블릭 키를 깃허브 서버에 올렸음
이제 SSH 키를 만들었던 컴퓨터는 깃허브 저장소의 SSH 주소만 알면 따로 로그인 정보를 입력하지 않고도 깃허브 저장소에 접속할 수 있음

SSH 주소로 원격 저장소 연결하기 - (1)

1. 우선 깃허브에 새로운 원격 저장소를 만들

깃허브 사이트에서 화면 오른쪽 위에 있는 [+]를 클릭한 후 [New repository]를 선택

저장소 이름을 입력한 후 [Create repository]를 클릭해서 저장소를 만드는데 이번에는 'test-2'라는 이름을 사용

2. 저장소가 만들어지면 기본적으로 HTTPS 주소가 나타남

우리는 SSH 방식으로 접근할 것이므로 [SSH]를 클릭해서 SSH 주소를 표시하고 SSH 주소를 복사

3. 퍼블릭 키도 추가했고 SSH 주소도 알아냈으니 지역 저장소를 연결할 수 있음

그렇다면 이제 새로운 지역 저장소를 만들어 봄

깃 배시 창을 열고 홈 디렉터리로 이동한 후 'connect-ssh'라는 디렉터를 지역 저장소로 만들고 connect-ssh 디렉터리로 이동

```
$ cd ~  
$ git init connect-ssh  
$ cd connect-ssh
```


SSH 주소로 원격 저장소 연결하기 - (2)

4. SSH 주소를 사용해 원격 저장소에 연결하는 방법은 HTTPS 주소를 사용할 때와 같음
git remote add origin 명령 뒤에 복사한 주소를 붙여 넣음
5. 오류 메시지 없이 프롬프트(\$)가 표시되면 정상으로 연결된 것
git remote 명령 뒤에 -v 옵션을 붙여서 어떤 원격 저장소가 연결되었는지 확인할 수 있음

```
$ git remote -v
```

6. 지역 저장소에서 f.txt라는 파일을 만든 후 숫자 '1'을 입력하고 저장함
그리고 스테이징과 커밋을 진행하는데 커밋 메시지는 'test ssh'라고 함
7. 이제 push 명령을 사용해 원격 저장소로 푸시

```
$ git push -u origin main
```

8. 깃허브 저장소에서 화면을 새로 고침 하면 방금 푸시한 파일이 올라와 있음
HTTPS와 SSH는 접속 방식만 다를 뿐 지역 저장소와 원격 저장소를 연결하는 방법, 푸시/풀하는 방법도 같음